# 0213. House Robber II

| Category: 1-D Dynamic Programming | LeetCode: https://leetcode.com/problem

# Problem

- Same as House Robber, but the houses form a circle: the first and last are adjacent.

# Intuition

- The first and last houses cannot both be robbed. So solve two linear problems: houses [0..n-2] and [1..n-1], and take the max.

# Brute Force

- Enumerate subsets with a circular adjacency check: O(2^n).

# Optimal Approach

- If n == 1 return nums[0].
- Return max(rob(0, n-2), rob(1, n-1)) using the linear solver.

# Worked Example

```
[2,3,2].

exclude last : [2,3] -> 3
exclude first: [3,2] -> 3
answer 3
```

# Java Solution

```java
public class HouseRobberII {
    public int rob(int[] nums) {
        int n = nums.length;
        if (n == 1) return nums[0];
        HouseRobber linear = new HouseRobber();
        return Math.max(linear.rob(nums, 0, n - 2), linear.rob(nums, 1, n - 1));
    }
}
```

# Complexity

- Time: O(n). Space: O(1).

# Pitfalls

- N == 1: both ranges would be empty, so special-case it.